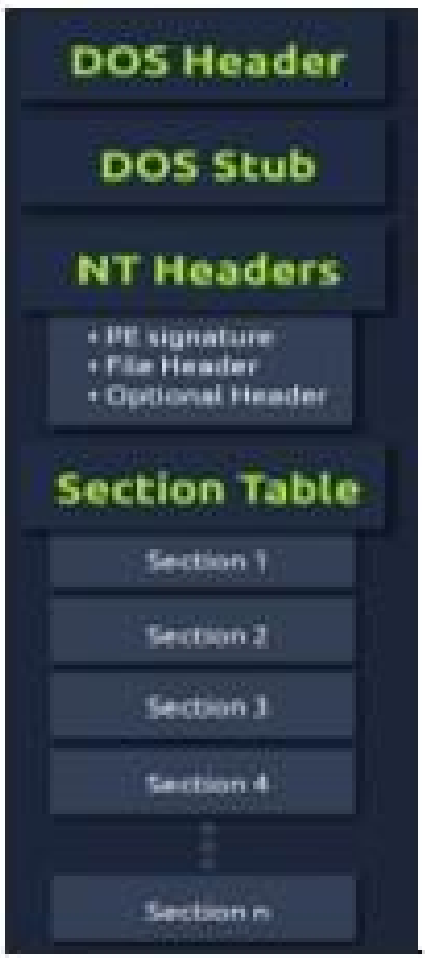
PE?

In the Windows Operating System, you might have often seen files with extension .exe. The .exe extension here stands for executable. As the name suggests, an executable file contains code that can be executed. Therefore, anything that needs to be run on a Windows Operating System is executed using an executable file, also called a Portable Executable file (PE file).

-THM

It's just a file format for windows



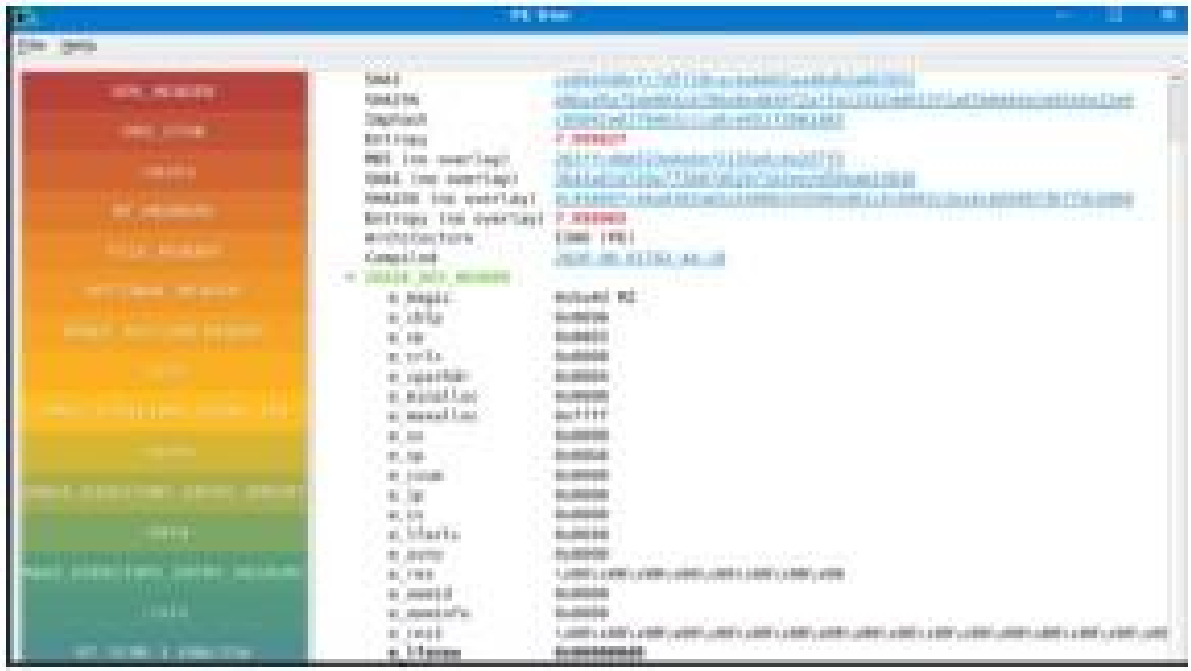
In total there are 6 headers of a Portable executable file each serving their own purpose

1. IMAGE_DOS_HEADER

it is the start of the pe file which contains the first 64 bytes of the file and the first 2 bytes always say MZ (mark zbikowski).

- MZ was the architect behind MS-DOS file format

- We have a drop down menu in each header and these are called values, we have given the values names so its just a label for a value just like e_magic (which is the value at the start of the header and display the "MZ"), e_cp , etc.
- The last value or we can say label is very important named "e_lfanew", it denotes the address where the IMAGE_NT_HEADER starts .

- 

as we can see where the IMAGE_NT_HEADERS starts at the very bottom.

NOTE : It says e_magic and has a value of 0x5a4d, 4D. This is the same as what we saw in the Hex Editor above, but the byte order is reversed due to endianness. The Intel x86 architecture uses a little-endian format, while ARM uses a big-endian format.

Something interesting to keep in mind .

2.DOS STUB

- Nothing much important here, it prints `"This program cannot be run in DOS mode"` if you try to run it on old DOS systems.
- Not much in use nowadays

3.IMAGE_NT_HEADERS

- Starts at the value of "e_lfanew" as mentioned above it is "The real start of the PE structure".

- **It consists of the following parts**
  1.Signature.
  2.FILE_HEADER .
  3.OPTIONAL_HEADER.

First of all, the term "offset"

- An **offset** just means **"how far something is from the start."**
  That's it.

  Think of a PE file (or even memory) like a long street of houses, all lined up in order.
  House #0 is the first one — that's your "base."
  If someone says "offset 200h," they mean *the house that's 200h (512 in decimal) bytes away from the start of the street.*

  let's start with

SIGNATURE:
The first 4 bytes of the NT_HEADERS consist of the Signature. If we open a file in HEX we will se 50 45 00 00, or the characters "PE" in ASCII.



Its just a signature that tells it's a PE file.

FILE_HEADER
The file header contains vital information
Machine: This field mentions the type of architecture for which the PE file is written.
NumberOfSections: A PE file contains different sections where code, variables, and other resources are stored. This field of the IMAGE_FILE_HEADER mentions the number of sections the PE file has.
TimeDateStamp: lol, the value can be changed easily if it's a malware , it just a build date and time of executable.

Characteristics:

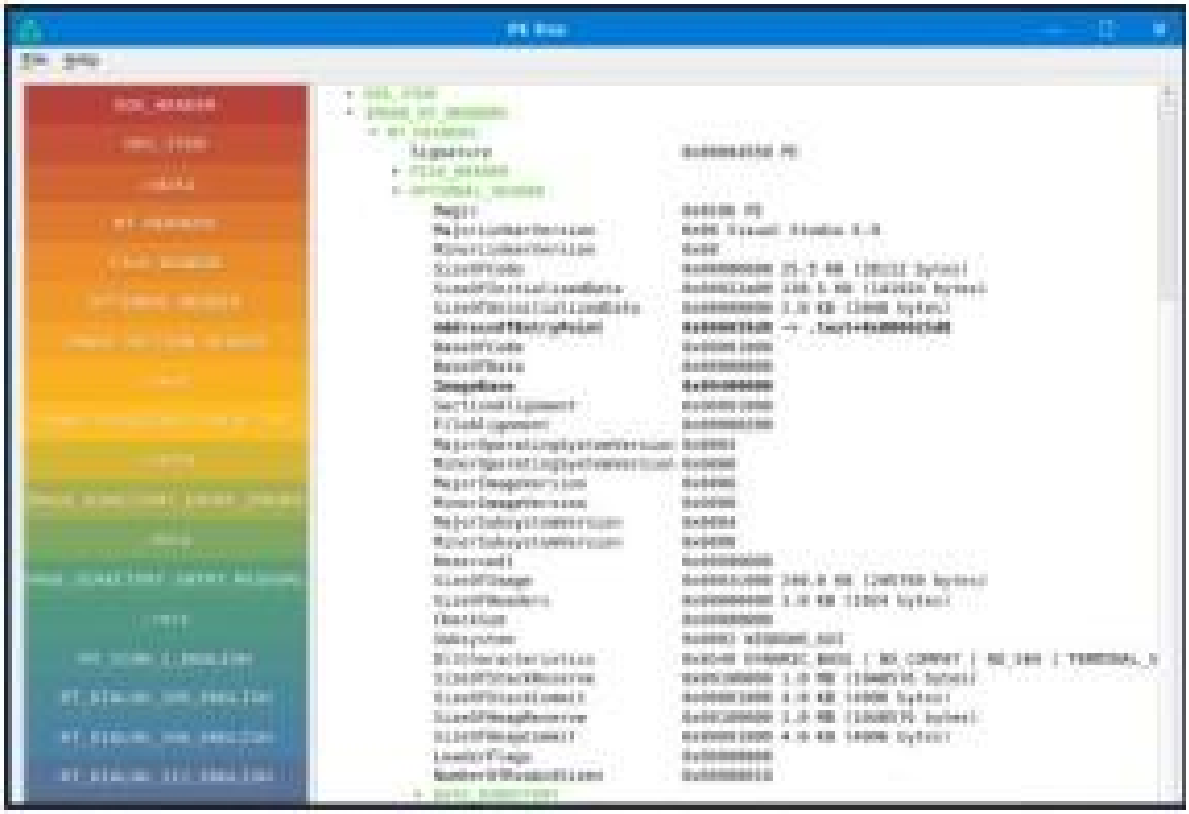
Let's say you have a PE file (like `mytool.exe`).
Windows needs to know:

- Is this a 32-bit program?
- Is it a DLL?
- Is it an EXE?
- Can it run on multiprocessors?

It tells characteristics of a PE file (no shit).

4.OPTIONAL_HEADER.

It is by no means optional, it is actually very important,



- **Starts at "Magic":** The value 010B Means it's a 32 bit PE file,if it was 64 bit file the value would have been 020B.

- **"AddressofEntryPoint":** This field is significant from a malware analysis/reverse-engineering point of view. This is the address from where Windows will begin execution. In other words, the first instruction to be executed is present at this address.
- **ImageBase**

The *starting address* in memory where the program **wants** to be loaded.

BaseOfCode & BaseOfData

They mark where your **code** (instructions) and **data** (variables, strings, etc.) start,

ImageBase = 0x00400000
BaseOfCode = 0x00001000

Then actual code starts at "`0x00401000`."

Subsystem

Tells Windows *what kind of program* this is:

Subsystem	Code	Meaning
0x0001	Native	Used by the OS itself
0x0002	GUI	Normal Windows app with a window
0x0003	CUI	Console app (like cmd.exe)

So your 0x0002 means. **Windows GUI app.**

DataDirector:

A table that points to important info inside the PE file like:

- **Imports** → which DLLs and functions it uses (e.g., `CreateWindowExW`)
- **Exports** → which functions it provides for others

- **Resources** → icons, dialogs, etc.

  You might be thinking **"WTH would a PE file export." (export means offer a functionality)**, well surprise surprise a PE file can be a EXE and a DLL.

Characteristic Meaning

"IMAGE_FILE_EXECUTABLE_IMAGE" ------->It's an EXE (runnable)

"IMAGE_FILE_DLL" -------> It's a DLL

"IMAGE_FILE_SYSTEM"-------> It's a driver

DataDirectory is important because as it gives a glimpse of what the PE file might be trying to do.

IMAGE_SECTION_HEADER:

The data that a PE file needs to perform its functions, like code, icons, images, User Interface elements, etc., are stored in different Sections. We can find information about these Sections in the IMAGE_SECTION_HEADER.



.text: The .text section is generally the section that contains executable code for the application. We can see above that the Characteristics for this section include CODE, EXECUTE and READ, meaning that this section contains executable code, which can be read but can't be written to.

.data: contains initialized variables (things with a starting value) for example "int x=10;" stuff like this

.rdata: Keeps **import info** (which Windows APIs or DLLs are used). Such as user32.dll for creating a file.

.ndata: it contains uninitialized variable for example "int x";

.reloc :Relocation info — helps Windows move the program if it can't load at its preferred address.

.rsrc: Resources — icons, dialogs, images, version info.

let's see what important information the section headers for each section include:

VirtualAddress: This field indicates this section's Relative Virtual Address (RVA) in the memory.

VirtualSize: This field indicates the section's size once loaded into the memory.

SizeOfRawData: This field represents the section size as stored on the disk before the PE file is loaded in memory.

Characteristics: The characteristics field tells us the permissions that the section has. For example, if the section has READ permissions, WRITE permissions or EXECUTE permissions.

IMPORT_IMAGE_DESCRIPTOR:

PE files don't contain all the code they need to perform their functions. In a Windows Operating System, PE files leverage code from the Windows API to perform many functions. The IMAGE_IMPORT_DESCRIPTOR structure contains information about the different Windows APIs that the PE file loads when executed. This information is handy in identifying the potential activity that a PE file might perform. For example, if a PE file imports CreateFile API, it indicates that it might create a file when executed.



Here we can see that the PE file we are looking at imports functions from ADVAPI32.dll, SHELL32.dll, ole32.dll, COMCTL32.dll, and USER32.dll. These files are dynamically linked libraries that export Windows functions or APIs for other PE files.



Its IMAGE_IMPORT_DESCRIPTOR imports notable functions such as CreateProcessW, CreateDirectoryW, and WriteFile from kernel32.dll. This implies that this PE file intends to create a process, create a directory, and write some data to a file. Similarly, by studying the rest of the imports, we can potentially identify other activities that a PE file intends to perform.